

Authenticate Me:

In this multi-phase project, you will learn how to put together an entire Express application with authentication. When you are finished, you will have a full project starter that you could use for creating your first API.

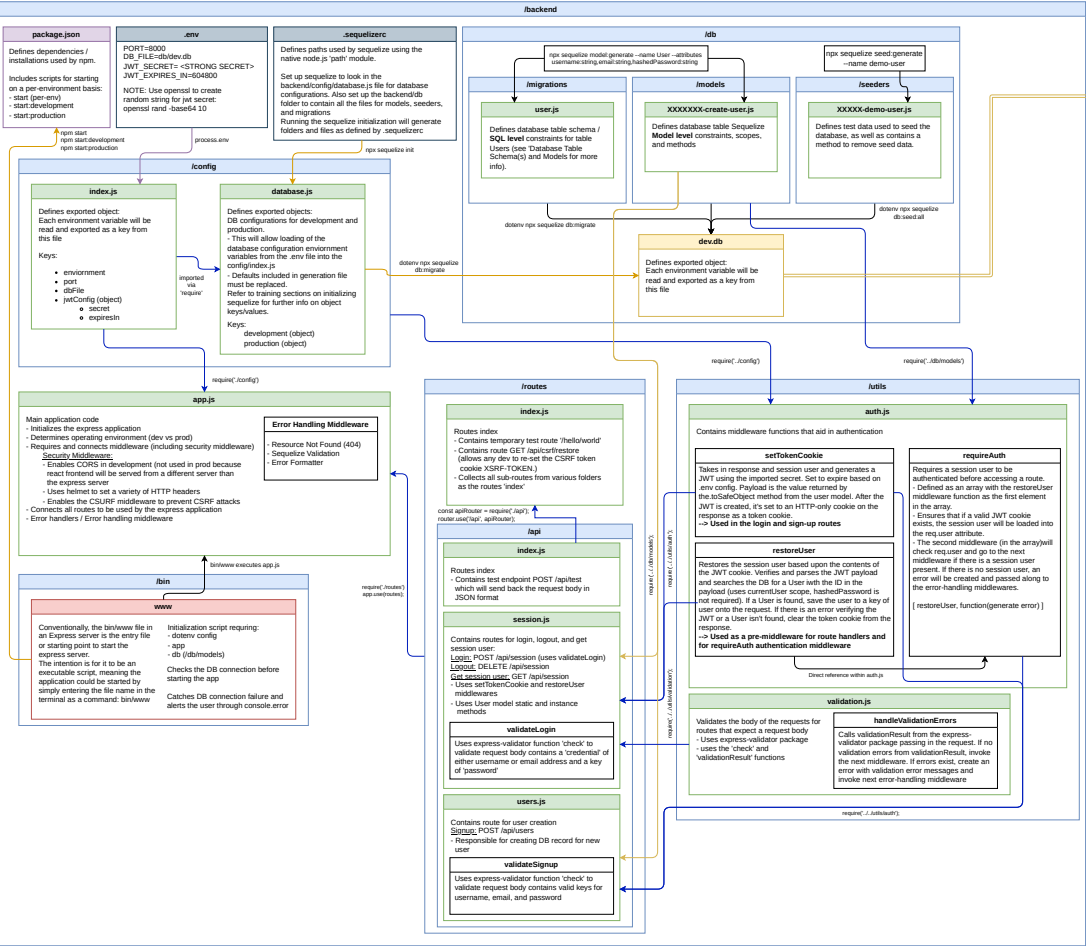
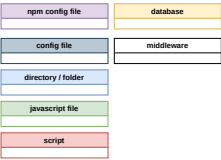
- Backend Phases
 - Phase 0: Set Up
 - Phase 1: API Routes
 - Phase 2: Error Handling
 - Phase 3: User Authentication
 - Phase 4: User Auth Routes
 - Phase 5: Validating the Request Body
- Deployment to Heroku

Dependencies

- cookie-parser - parsing cookies from requests
- cors - CORS
- csrf - CSRF protection
- dotenv - load environment variables into Node.js from a .env file
- express - Express
- express-async-errors - handling async route handlers
- helmet - security middleware
- jsonwebtoken - JWT
- morgan - logging information about server requests/responses
- per-env - use environment variables for starting app differently
- qs - PostgresQL greater or equal to version 8.4.1
- sequelize - Sequelize
- sequelize-cli - use sequelize in the command line
- bcryptjs - user password salting / hashing
- express-validator - used to validate the body of requests for routes

Dev Dependencies

- dotenv - use dotenv in the command line
- nodemon - hot-reload server backend files



Schema			Model - Validations		
Column Name	Data Type	Constraints	Column Name	Data Type	Validations / Constraints
id	integer	not null, primary key	username	string	not null, unique, min 4 characters, max 30 characters, isEmail
username	string	not null, indexed, unique, max 30 characters	email	string	not null, unique, min 3 characters, max 256 characters, isEmail
email	string	not null, indexed, unique, max 256 characters	hashedPassword	binary string	not null, min and max 60 characters
hashedPassword	binary string	not null			
createdAt	datetime	not null, default value of now()			
updatedAt	datetime	not null, default value of now()			
Send Data			Model - Scopes		
email	username	hashedPassword	Name		Details
demoUser	demoUser	bcrypt.hashSync(password)			Used by default when a query is made without specifying a scope.
user	user	bcrypt.hashSync(password)	defaultScope		attributes: [exclude: ['hashedPassword', 'email', 'createdAt', 'updatedAt']]
user2	user2	bcrypt.hashSync(password)			Returns all fields except the password
user3	user3	bcrypt.hashSync(password)			attributes: [exclude: ['hashedPassword']]
					Used for queries to verify user login credentials. Returns all fields
					attributes: []
Model - Methods					
Name	Type	Details			
toSafeObject	Instance	Returns an object with only the User instance information that is safe to save to a JWT (like id, username, and email)			
validatePassword	Instance	Accepts a password string and will return true if there is a match with the User instance's hashed password. Otherwise, return false. Uses bcrypt.			
getCurrentUserId	Static	Accepts an id. Uses the currentUser scope to return a User with the specified id.			
login	Static	Accepts an object with username and password keys. Searches for one user with the specified credentials (username or email). If a user is found, validates the password by passing it into the instance's validatePassword method. If the password is valid, return the user by using the currentUser scope.			
signup	Static	Accepts an object with username, email, and password keys. Hashes the password using bcrypt.hashSync. Creates a user with the username, email, and hashedPassword. Returns the newly created user using the currentUser scope.			

Authentication Login Flow

- The API login route will be hit with a request body holding a valid credential (either username or email) and password combination.
- The API login handler will look for a User with the input credential in either the username or email columns.
- Then the hashedPassword for that found User will be compared with the input password for a match.
- If there is a match, the API login route should send back a JWT in an HTTP-only cookie and a response body. The JWT and the body will hold the user's id, username, and email.

Sign-up Flow

- The API signup route will be hit with a request body holding a username, email, and password.
- The API signup handler will create a User with the username, an email, and a hashedPassword created from the input password.
- If the creation is successful, the API signup route should send back a JWT in an HTTP-only cookie and a response body. The JWT and the body will hold the user's id, username, and email.

Logout Flow

- The API logout route will be hit with a request.
- The API logout handler will remove the JWT cookie set by the login or signup API routes and return a JSON success message.